

Batcher: Declared Execution Contracts Across Cores, Clusters, Streams, and Models

Stephen Offer

Anyscale • stephen.offer@anyscale.com

Abstract

Data engines usually cross the line between one node and a cluster, or between bounded and continuous input, by adding an execution path on the far side, and then must keep the paths in agreement. Batcher instead admits a stateful operator only if it declares an execution contract, and its distributed scheduler dispatches on that contract. **C1** operators merge partial states under a commutative monoid. **C2** operators run independently on each partition of a declared hash or range exchange. **C3** operators have no partitioning key: they replicate a bounded input or iterate ordinary plans under a round cap, and refuse past either budget. A registry classifies every IR operator and aggregate, and the test suite holds it exhaustive. The micro-batch path and the model-inference operators, which are plan expressions rather than opaque callbacks, run under the same contracts. On one 48-core machine reading the same Arrow buffers, Batcher beats DuckDB on six of six suites (geometric-mean ratios 0.16 to 0.83). Against DuckDB’s own compressed store it leads five (0.35 to 0.75) and is at parity on the sixth. At TPC-H sf100 on a 100-worker fleet it leads Daft on three pipelines by 3.7 to 109 $\times$. The latency wins cost CPU (1.86 $\times$ DuckDB’s on TPC-H sf1), and Batcher loses the Join Order Benchmark by 11%.

1 Introduction

Ask a database engineer how `GROUP BY` works and you often get two answers: a hash table on one machine, and on a cluster a partial pass followed by a shuffle and reduce. Many stacks build these as separate operators in a single-node engine and a distributed one, often beside a stream processor and a model-serving runtime with their own `GROUP BY` or notion of a user function. Some systems avoid the split. Flink runs bounded input as a special case of one streaming runtime [12], and the Dataflow model gives batch and streaming one specification [1]. Spark SQL sends its local and distributed modes through the same physical operators [2]. None of them makes a declared execution contract the only route by which a stateful operator enters the engine. This paper tests that design.

Batcher’s own history shows why the route matters. A floating-point grouping key grouped correctly in the local hash table and split into two groups after a distributed shuffle. The single-node suite stayed green throughout, since that path was correct. Nothing forced the two paths to agree.

1.1 What changed

Most organizations run a distributed stack designed between roughly 2009 and 2014 for conditions that have since moved. Spark starts from the premise that the data does not fit in one machine’s memory [50]. The single node measured here has 48 hardware threads and 92 GiB, which is not large, yet a query that takes milliseconds in-process can take seconds through a distributed runtime’s dispatch. Distribution therefore has to become a property of the query rather than of the engine. Freshness went from nightly to continuous, so bounded input is no longer the only case an engine plans for. Object storage, schema drift and opaque operators (a regular expression, a JSON extraction, an embedding model, an LLM call) took away the curated statistics a static optimizer prices from, which argues for measuring instead (Section 6). Models moved into the middle of pipelines. They read decoded media and run on a device, then feed a join on CPU (Section 5). Practitioners write Python, where the usual escape hatch is a user function the optimizer cannot see into (Section 4). IDC projects that most enterprise data is unstructured [41], a forecast rather than a census; either way, an engine whose type system stops at scalars must hand an image or a clip to something else.

1.2 The admission rule

Batcher responds by constraining its operator vocabulary instead of adding an execution layer per regime. A stateful operator enters only with one of three contracts (Section 3). **C1** is a mergeable partial $\rightarrow$ combine $\rightarrow$ finalize triple. **C2** evaluates partition-locally after a declared hash or range exchange; the range form also needs a total order and a tie policy. **C3** covers operators with no partitioning key at all. Its replicate and iterate forms each carry a budget, and a breach raises an error instead of falling back to one node. A registry maps every relational operator and aggregate in the IR to its contract and a declared state class. Adding an IR tag without an entry fails a unit test.

The contracts form a scheduling interface, not a compactness theorem. **C1** admits `DISTINCT` and `array_agg`, whose state grows with what they retain, and Table 2 names such operators instead of hiding them behind one state bound. Every contract does make the partition count a scheduling variable and nothing more. It can reorder a floating-point sum and, in the four cases SQL leaves open, change which permitted answer comes back (Section 3.5). The relation itself does not change.

The micro-batch path uses the same vocabulary: a **C1** operator folds each trigger’s partial into its running state. Media

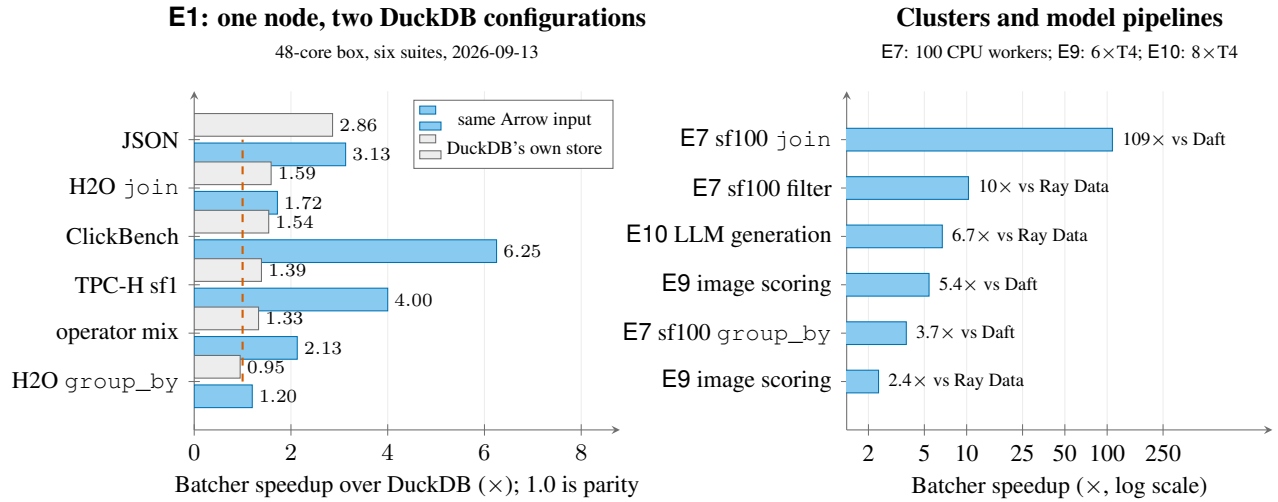


Figure 1: Headline results, tagged by experiment; Table 3 binds each tag to its hardware and date and states the protocol. Left: six suites against DuckDB on one machine, blue on the identical zero-copy Arrow buffers Batcher reads, grey after an untimed ingest into DuckDB’s compressed, zone-mapped store. The grey bars pit DuckDB’s storage and execution against Batcher’s execution alone, and Section 8.3 explains why that isolates neither. Right: separate experiments on different fleets, on a log axis, not one matched experiment. Every bar passed a correctness gate before it was timed.

decode is a plan expression too, as are embedding and model inference, so Kyber (the optimizer) and Carbonite (the resource manager) reason about them alongside the joins around them.¹

1.3 Contributions

1. **Three execution contracts enforced as an admission rule.** Each states a merge or partitioning law, a state class and a failure mode, and a registry classifies the whole IR against them (Section 3). The algebra is old. Enforcing it as the only admission route is the design choice under test.
2. **A stated equivalence relation** between execution modes, exact on the relation except for floating-point reassociation and the four SQL-open choices. Integer SUM overflow does not depend on the partitioning (Section 3.5).
3. **Measurement-driven planning under a result-invariance rule:** a stale measurement costs time and never changes an answer. An ablation shows which part of the repeated-query speedup it buys (Sections 6 and 8.7).
4. **Relational and model operators in one plan,** under one optimizer and one resource manager (Sections 4 and 5).
5. **A traceable evaluation:** every number carries an experiment tag bound to hardware, date, lineup and statistic (Table 3). Storage configurations, CPU cost, memoized answers and worker loss are reported separately (Section 8).

Reading the numbers. A ratio written b/x means $\text{Batcher}_{ms}/x_{ms}$ for baseline x , so **below 1.00 means Batcher is faster**. Suite ratios are geometric means over the included queries; the bar charts invert them into speedups. DuckDB runs in the two configurations Figure 1 defines, *same Arrow* and *native store*, never netted together.

¹The engine is Apache-2.0 at <https://github.com/stephenoffer/batcher>, as are the benchmark harness and every comparator adapter. Appendix C lists the commands behind every experiment.

2 System overview

Batcher puts a Python control plane over a Rust data plane. Arrow `RecordBatch` [44] is the only columnar format, at every internal boundary and across the FFI (Figure 2). Python builds a lazy plan, optimizes it and lowers it to a JSON IR. The Rust engine executes the IR and returns Arrow batches through the C Data Interface. The control plane pays per query, so its seven optimizer phases fit in single-digit milliseconds (Section 8.5). The data plane pays per row.

The API surface. Users meet Batcher at four layers, and SQL included, all of them lower to the same `LogicalPlan` (Table 1). A `Dataset`, the lazy and immutable handle to a plan, has 153 public members, from relational verbs and windows to terminals such as `collect`, `iter_batches` and `write`. The scalar language, `Expr`, has 191 methods and eleven accessor namespaces (`.str` alone has 186). Feature surfaces sit on top: `ds.ml` for models, `bt.graph` for graph analytics over an edge table, `ds.dq` for data-quality contracts, and a security catalog whose row filters and column masks are plan rewrites. Streaming needs no separate API. Give a sink a `trigger`, `output_mode` or `checkpoint` and the same plan runs continuously.

The Python packages. An import-linter contract enforces the control plane’s layering on every commit. Only `api` imports the four subsystems, which it sequences at a terminal call. Kyber rewrites plans and never executes. Carbonite admits a plan against a memory envelope, reserves memory and hands out shuffle credits, but never rewrites. Core executes and measures. Governance rewrites plans for row and column policy. Because the four may not import one another, a helper two of them need goes in a neutral layer: `plan` (the IR and schema, plus the contract registry), `metadata` (the Metadata Hub), `io`, `interop`

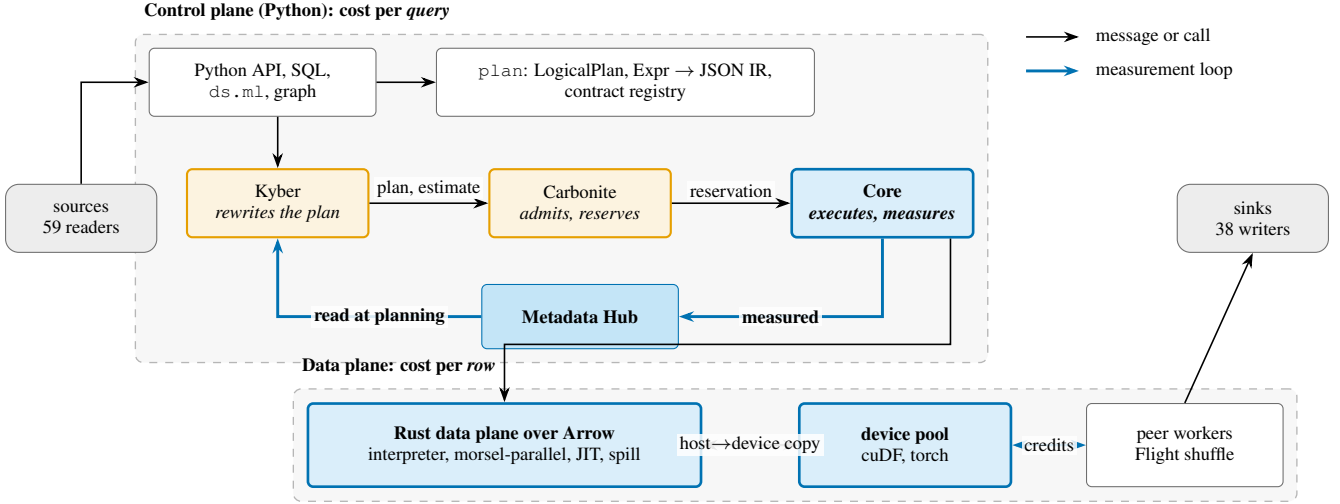


Figure 2: The whole engine. Arrows show messages and data flow, not source imports; a lint contract checks on every commit that Kyber, Carbonite and Core never import one another. Relational and expression work runs in Rust over Arrow and returns through the Arrow C Data Interface, a pointer exchange inside one process. Three parts of the data plane are not Rust: a user’s batch callback, a model framework’s own code, and the cuDF device tier, which copies its inputs from host to device memory. The peer-worker edge crosses a network, with framing and real copies. Measurements reach planning only through the Metadata Hub.

Layer	What it contains
Entry points	bt.read.* (59 readers: files, media, genomics, lakehouse tables, 11 databases, 9 stream sources); from_pandas/polars/arrow/torch and 20 other constructors; bt.sql, Session
Dataset	select, filter, with_columns, join (equi, as-of, inequality, lookup), group_by/agg, cube/rollup, set operations, sort, top-k, pivot/unpivot, explode, windows, map_batches; 38 ds.write.* sinks incl. Delta, Iceberg and merge_into
Expr	191 methods; namespaces.str.list.dt.json.map.struct.seq.meta and the media namespaces.image.audio.video; 672 top-level names incl. geospatial st_* and rigid-body quat_*/se3_*
Feature surfaces	ds.ml (infer, embed, generate, iter_torch_batches, similarity join); bt.graph (60 algorithms); ds.dq; ds.sed and CDC; governance and lineage
Streaming	a sink with trigger/checkpoint; with_watermark, session_window, join_stream, transform_with_state
Operations	Config (15 sections), config_context; start_ui, query history, cancellation; 17 typed errors rooted at BatchError

Table 1: The public API by layer, as counted in the installed package. All of it lowers to one LogicalPlan and one JSON IR.

or config. dist schedules the same operators on Ray and may not import api.

The Rust crates. Fifteen crates form a one-way dependency graph. bc-arrow defines the morsel: 16,384 rows or 1 MiB, whichever comes first. bc-expr and bc-ir define the one scalar expression type and the one relational plan type. bc-runtime keeps the mergeable state for aggregation, joins, shuffle partitioning, top-N and windows. bc-codegen compiles a subset of Expr with Cranelift [11]. bc-interp contains the sequential interpreter (the correctness oracle) and the morsel-parallel, streaming and spilling executors. The Arrow Flight shuffle [45] is bc-transport, with the sketches in bc-sketches and the native Parquet reader in bc-io. Only bc-py links Python. The geometry and rigid-body crates feed bc-expr.

Execution tiers. The same plan can run on the sequential interpreter or the morsel-parallel executor [29], with Cranelift-compiled expressions [37], out of core through the spilling executor, across a Ray cluster, or with cuDF on each GPU. The first five share every kernel. On the subset it compiles, the JIT must match the interpreter bit for bit, and elsewhere it falls back. A fuzzer compares raw result bits, and the suite records compile-versus-fallback counters. The cuDF tier cannot share the Rust code and works as a translator instead. It translates each IR tag or declines it with a reason, and a test holds that classification exhaustive. Before a device result is returned, the engine checks it against its static schema (Section 7).

3 Three contracts

The distributed scheduler reads one fact about a stateful operator: its contract. Table 2 gives the classification, and Figure 3 maps the first two contracts onto schedulers.

3.1 C1: mergeable

An operator satisfies C1 when it factors into partial: $\mathcal{M}(R) \rightarrow S$, $\oplus: S \times S \rightarrow S$ and finalize: $S \rightarrow \mathcal{M}(R')$, where $\mathcal{M}(\cdot)$ is a multiset, $(S, \oplus)$ is a commutative monoid with identity ε , and partial is a monoid homomorphism from $(\mathcal{M}(R), \oplus)$. Then for every partitioning $\{p_i\}_{i=1}^n$ of an input multiset P into disjoint parts whose union preserves multiplicity,

$$\text{finalize}\left(\bigoplus_{i=1}^n \text{partial}(p_i)\right) = \text{finalize}(\text{partial}(P)). \quad (1)$$

Equation (1) follows from the homomorphism and the monoid laws. It is sufficient, and we claim nothing about necessity. A property test checks it against the single-pass result over randomly drawn cut points, empty and single-row parts included, which gives evidence rather than proof.

C1 is a scheduling interface and says nothing about size.

Any deterministic bag-to-bag query satisfies Equation (1) with $\text{partial}(P) = P$ and multiset union as $\oplus$, running the whole query in finalize. A bound $|S| \in O(gw)$ does not exclude that construction either if g may be any key the implementation chooses: keying by the whole row gives $g = |P|$ on distinct input. Batcher therefore does not equate C1 with compact state. Each C1 operator instead declares one of three *state classes*, checked against the query rather than the implementation:

$$|S| \leq \begin{cases} g_Q \cdot w_{op} & \text{FIXED: } g_Q = \text{groups of the query's key} \\ \sum_{e \in \text{retained}} |e| & \text{RETAINS: entries the result needs} \\ s_{cfg} & \text{SKETCH: configured size} \end{cases} \quad (2)$$

Here g_Q counts the distinct values of the grouping key the *query* names, and w_{op} is a per-operator constant (three machine words for variance, two for AVG). That rules out the union trick for every FIXED operator. A SUM grouped by one column of a ten-column row cannot key its state by the whole row without exceeding $g_Q w$ on any input where rows outnumber groups. RETAINS is the honest class for operators whose answer needs what they keep: DISTINCT over a unique column keeps every value, array_agg over one group keeps the input, and exact median keeps each group's values to select at finalize. A hash-join build keyed on one value repeated a million times keeps a million payload rows, because its entries are build rows, duplicates included. For these the exchange carries the entries, the merge concatenates or unions, and spilling rather than algebra bounds memory (Section 3.6).

What an implementer supplies. The signature $S \times S \rightarrow S$ hides four obligations. First, two states may combine only when their key types, aggregate list and parameters (a quantile's q , a sketch's precision) agree. Each partial carries a fingerprint of these in its Arrow schema metadata, and the receiver raises a typed error on a mismatch instead of merging silently. Second, ε need not be the SQL answer on empty input (empty COUNT is zero, empty SUM is null), so every accumulator carries a validity flag. Third, finalize is not always a projection: AVG divides, variance corrects, a quantile selects. Fourth, state crosses the network as Arrow IPC, and a round-trip test covers values, validity, keys and the fingerprint. AVG is the standard trap. Averaging partial averages works on equal partitions and fails on unequal ones, which is what a cluster produces, so the state is (Σ, c) and the property test draws deliberately lopsided cuts.

Sketches. Register sketches merge identically in any order: HyperLogLog takes a register-wise maximum, Count-Min a cell-wise sum, Bloom a bitwise OR. The approximate-quantile aggregates use DDSketch, whose state is a set of logarithmic bucket counts merged by bucket-wise addition. Equation (1) thus holds exactly on the state, and the approximation enters only in finalize, as a relative error α fixed by configuration. The optimizer's column statistics use a KLL sketch [25], whose merge compacts and depends on order. It never computes a query answer. Its tests check the rank-error bound instead of state equality.

3.2 C2: partition-local after a declared exchange

A C2 operator runs independently on each partition of a declared exchange, and the outputs are concatenated. A *hash* exchange routes rows by a hash of a declared key, so all rows sharing a key value meet on one worker. Equi-joins (outer joins included) and windows with a PARTITION BY run this way. A right or full join always co-partitions, even where the planner would have broadcast, so the one worker holding every row with a key decides whether a build row matched and emits an unmatched row there once. A *range* exchange routes rows by a total order on a declared key, with boundaries drawn from a merged sample. Sort and the globally ordered window functions use it.

Two more declarations complete the range form. Ownership is half-open: a key equal to a split point belongs to the upper range, so every copy of a key value lands in one range and no tie crosses a boundary. Nulls go to a declared end. Rank-like functions then need only per-range summaries. In range j , rank adds the row count of ranges $0..j-1$ and dense_rank adds their distinct-key count, while lag exchanges each range's tail. A split at a heavily repeated key therefore gives the same ranks as one node. Tie order under row_number and in a distributed sort is unspecified, while top- N breaks ties by input position and is deterministic. Under distribution, Batcher refuses with a PlanError any global window it cannot summarize this way (lead, exact median, count_distinct, fills, an explicit frame, a computed ordering key).

3.3 C3: no partitioning key

C3 covers operators with no key at all, in two forms with separate obligations. *Replicate* broadcasts one input to every worker and partitions the other, and Batcher runs inequality and band joins this way. It is exact for the join types that decide a left row's match set within that row's partition (inner, left, semi, anti). Right and full inequality joins are not offered. The broadcast side is always the right input, under a byte budget derived from cache size and scaled for distribution. Over budget, the planner raises with the reason. An empty right side is not refused: it short-circuits to the answer each join type defines. Replication is this implementation's choice: the block-pair pruning of IEJoin [26] would partition both sides, and Batcher does not implement it.

Iterate runs a recursive common table expression as a control-plane loop of ordinary plans, each dispatched by contract. The accepted grammar is anchor UNION [ALL] step, where the step references the CTE exactly once (twice is refused) and reads only the previous round's new rows. Under UNION, each round removes rows already seen with a null-safe set difference, so a cycle terminates once it stops producing new rows. UNION ALL removes nothing. A loop still producing rows after 1,024 rounds raises; the cap is a resource policy and proves nothing about termination.

Both forms obey one law, which makes C3 a contract rather than a list of exceptions. Execution is legal only under a declared budget (bytes for replicate, rounds for iterate), and exceeding it raises. Silently running the whole operator on one node instead would hide a missing distributed path behind a latency cliff and

Operator or aggregate	C	state	$\oplus$ / finalize
count sum min max avg, bool/bit, product, any_value, arg_min/max variance, stddev, covariance, correlation, skew, kurtosis array_agg, list_agg (ordered or not)	C1	FIXED	add or compare; AVG divides
count(distinct), mode, approx_top_k	C1	RETAINS	concatenate; order by key if given union with counts
exact median, quantile, mad, histogram, entropy	C1	RETAINS	concatenate; select at finalize
distinct	C1	RETAINS	set union
top- N (sort with a small limit), OVER () window	C1	FIXED	merge heaps; ties by input position
HLL, Count-Min, Bloom, DDSketch quantiles	C1	SKETCH	max / sum / OR / bucket add
equi-join, incl. outer window with PARTITION BY; as-of join by key sort; ordered global window LIMIT, row id	C2 hash C2 hash C2 range C2 range	RETAINS RETAINS RETAINS —	build rows incl. duplicates per-partition evaluation concatenate; rank offsets split by source position
equi-join, small build side	C3 replicate	budget	broadcast; over budget, shuffle
inequality / band join recursive CTE	C3 replicate C3 iterate	budget 1,024 rounds	broadcast right, or raise linear step, or raise

Table 2: The stateful vocabulary by contract and state class, as the registry of Section 3.4 records it. The test that keeps the registry exhaustive also keeps this table complete. Where a tag has several rows, the planner chooses among those contracts by size and shape. RETAINS state grows with what the answer needs, and spilling bounds it in memory (Section 3.6).

an out-of-memory risk.

3.4 Admission, and what it rejects

A registry in the neutral `plan` layer enforces the classification. It maps each relational IR tag and each aggregate function to its state class and to its contract, or to the set of contracts its distributed paths may take. A unit test parses the Rust IR enums and fails when a tag is unclassified, classified twice, or classified but gone from the IR. An operator added to the IR without a declared contract therefore fails the build’s test gate. That has already happened once. When the registry was merged onto newer engine code, the test failed on four aggregates added in the meantime (`arg_min_null`, `arg_max_null`, `modes`, `skewness_pop`), and each had to be classified before the build passed. The registry does not yet drive dispatch itself; the executor’s own routing does, and the test keeps the two in step.

The rule rejected two designs, neither on grounds of impossibility. A hash join whose build side is one table shared across workers has coherent semantics but no merge we were willing to synchronize, so the build side is partitioned or replicated instead. Streaming state with tombstones has no inverse in Batcher’s partial log. Richer incremental algebras such as DBSP [10] express it with signed updates, but adopting one would mean a fourth contract, which the rule forbids until someone argues for it.

3.5 The equivalence relation between modes

“The same result on one core and a hundred nodes” needs a definition. Under any partition count and any execution mode (sequential, morsel-parallel, distributed, micro-batch), Batcher

promises a result *equivalent* to the single-node one. Column names and types are equal, and so are the row multisets, except in four cases where SQL itself leaves the answer open.

1. *Floating-point aggregates* agree up to reassociation. IEEE addition is not associative, and partition count, morsel boundaries and micro-batch boundaries all change the summation tree, so batch and micro-batch runs over the same input are not bit-identical either.
2. An *unordered LIMIT n* returns exactly n rows, each a row of the unlimited result, but which n is unspecified. On rows $\{a, b\}$, `LIMIT 1` may return either. With an `ORDER BY` the answer is defined and both paths compute it.
3. *Ties in row_number and in a distributed sort*: the partitions, their sizes and the set of numbers are equal, but which tied row receives which number is not. Peers share a value under `rank` and `dense_rank`, so neither is affected.
4. An *unordered array_agg* returns the same elements as a multiset, in an unspecified order; `array_agg(order_by=...)` is deterministic.

The oracle tests check exactly this relation: an order-insensitive multiset comparison with a float tolerance for unordered results, an ordered comparison wherever the query orders, a subset-and-count check for unordered `LIMIT`, and a rank-set check for tied `row_number`.

The relation covers errors in one case. Integer `SUM` accumulates in `i64` and carries its *partial* state as a 128-bit integer, checking only the *final* total against the `Int64` result type. A partition whose own total overflows 64 bits thus no longer errors when the grand total fits: $M + 1 + (-1)$ for the largest representable M gives M under every grouping. Decimal `SUM` over `DECIMAL( $p, s$ )` returns `DECIMAL(38,  $s$ )`, as DuckDB does, and raises only past 38 digits. Decimal `AVG` and the moments run in `float64`. Which of two data errors a query reports first is unspecified, but predicate reordering cannot introduce one. The executor reorders only conjuncts it can prove infallible, so a filter that shields a strict cast keeps shielding it.

Reassociation also composes, which a tolerance on the final column misses. A last-bit difference can cross a `HAVING` threshold or become a different integer through `rank()`. TPC-DS q67 ranks a `SUM` over a `ROLLUP`: cast to `float64`, a last-bit disagreement produced different ranks, while on the source `DECIMAL` types all 100 rows match exactly and in order. Floating-point variance, covariance and the higher moments use Chan’s pairwise merge with Neumaier compensation, which reduces cancellation error without bounding it away. Plain floating `SUM` and `AVG` skip compensation for speed. Bitwise-reproducible summation under a free partition count is available, at a cost Batcher has not paid [17].

3.6 What the contracts bound, and what they do not

The executor streams every query, batch included. Operators pull morsels through pipelines and materialize only at pipeline

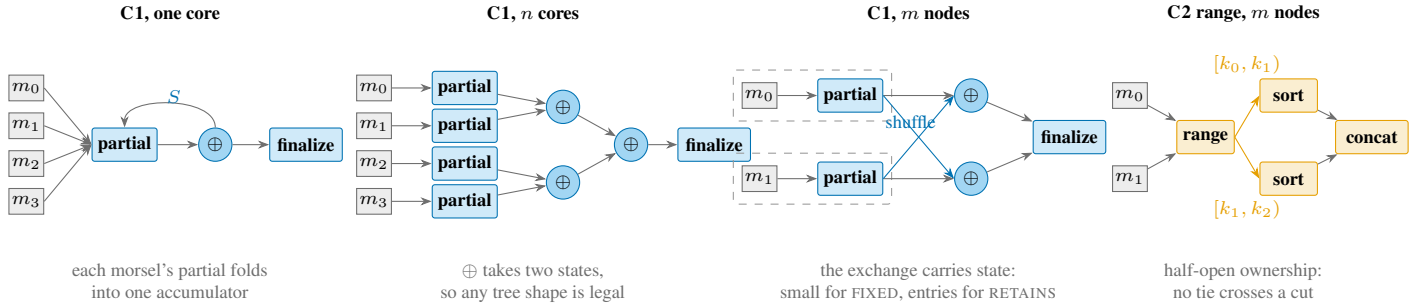


Figure 3: How the partitionable contracts map onto schedulers. Under C1 only the arrangement of `partial`, $\oplus$ and `finalize` changes: one core folds every partial into one accumulator, n cores combine up a tree, and m nodes exchange state. Associativity permits any tree shape, commutativity any arrival order. The state class sets what the exchange carries: 100 accumulators per worker for a 100-group SUM, the input itself for a DISTINCT over a unique column. The fourth panel shows the range form of C2: range-partition, sort, concatenate, with rank offsets computed from per-range counts.

breakers [29]. The memory Carbonite governs on one node is

$$M_{\text{node}} \leq \sum_{b \in \text{breakers}} R_b + \sum_{q \in Q_{\text{node}}} B_q + M_{\text{store}}, \quad (3)$$

where R_b is breaker b 's *resident* state, B_q the bytes in flight on transport queue q , and M_{store} the published shuffle output a worker holds for its peers. Each term has its own control. The memory envelope bounds R_b : past it, aggregation, distinct, sort, join build and partitioned windows spill, so a RETAINS operator's *logical* state (Equation (2)) can exceed memory while its resident state does not. M_{store} is capped at a quarter of RAM and spills beyond it. Credits bound B_q . A producer charges $\lceil \text{bytes}/\beta \rceil$ credits for an encoded batch, where β is the 1 MiB morsel byte cap, so a one-row morsel holding a 100 MiB tensor consumes most of the window instead of one slot. Capping the charge just below the window (9 of the default 16 credits) lets an oversized batch always be admitted, and gives $B_q \leq c\beta + r_{\text{max}}$ for a window of c credits, with r_{max} the largest single batch. Until this work, a gRPC message ceiling of 4 MiB made any larger batch unfetchable. The ceiling is now 2 GiB, and the credit window, not the ceiling, bounds memory. A per-gather byte budget (768 MiB, divided among its streams) times the number of concurrent gathers bounds receive-side buffering separately.

Equation (3) bounds executor-managed memory, not process RSS. Reader decode buffers, Arrow buffers kept alive by a slice, Python objects and a model framework's allocations fall outside it; Carbonite watches process RSS as a pressure signal but reserves nothing for them. Device memory has a separate pool, outside the envelope, that tracks model weights and KV caches. Nor is the number of transport queues constant. A gather opens $\max(48, \text{peers})$ streams, so past 48 peers the per-node queue term grows with the cluster, and only the gather budget stays fixed per node. Reader batch size once proved the point. Parquet read batches were sized in rows with no byte term, so 65,536 rows of decoded images became a 9.6 GB allocation that no downstream credit could see. The reader now targets 16 MiB per decoded batch.

A Rust test with a counting allocator measures the buffering term directly. It runs the TPC-H q3/q4/q5 join shape with the build side fixed and the probe side growing, and records peak

additional heap with inputs excluded: about 1 MB at one and at four million probe rows, where the materializing executor it replaced grew from 221 to 854 MB. That holds only where operators preserve or reduce rows. A high-fan-out join once produced one output batch per input morsel however large, and a skewed equi-join reached 13.1 GB RSS. Morselizing the join's output and slicing its probe input brought the same query to 728 MB with identical results. Forty-six passing oracle tests missed that defect, because a correctness suite cannot see a memory property.

3.7 The incremental form is the same state transition

A C1 operator that folds each batch through `partial` and $\oplus$ never reads its whole input, as a continuous query requires. Batch and micro-batch operators run the same kernels and differ only in whether `finalize` runs once or per trigger. That claim concerns kernels alone. A continuous query also needs output-update semantics, progress tracking, late-data handling and coordinated recovery, which Batcher has only in the micro-batch form described here; Section 9 compares it with Flink.

The commit protocol. Each trigger has a stable `batch_id`. The driver pulls and executes the batch without publishing anything, then records the source offsets for that `batch_id` in a write-ahead log. It writes the sink, then operator state, and marks the batch committed last. A crash before the commit replays the batch under the same `batch_id`, so correctness on replay falls to the sink. Delta commits carry a transaction marker (`app, batch_id`) checked before and after the write. Iceberg appends carry the same pair in the snapshot summary, and a table property also keeps the latest committed version so snapshot expiry does not erase it. A file sink names its output by batch. A keyed database sink upserts. Each of these makes a replayed batch a no-op. An unkeyed append to a plain database table does not, and is documented as at-least-once. When the micro-batch itself runs distributed, only the Delta publish carries the transaction check today, so Batcher refuses a distributed streaming write to Iceberg rather than silently downgrading it.

```

import batcher as bt

# 'path' and 'width' come from the reader, not from
# the pixels: the glob yields one row per file with
# its URI and JPEG header fields already populated.
frames = (bt.read.images("s3://clips/**/*.*.jpg")
          .with_columns(sku=bt.col("path")
                        .str.extract(r"/sku=(.+)/"))))
cat = bt.read.parquet("s3://catalog/")

hot = (frames
      .filter(bt.col("width") >= 640) # header, not pixels
      .with_columns(t=bt.col("image")
                    .image.to_tensor(224, 224))
      .ml.infer("resnet50", column="t",
                output_column="label") # a GPU operator
      .join(cat, on="sku") # C2: hash exchange
      .group_by("label") # C1: merge
      .agg(n=bt.col("sku").count()))

hot.collect(distributed="auto") # the scale knob

```

Figure 4: A Batcher program. Decode and inference share one logical plan with the join and the aggregation. The reader fills the JPEG header field width without decoding, so the optimizer can sink the filter below `image.to_tensor`. A filter on decoded pixels would have to stay put. Only the model framework’s own forward pass runs Python over data here, on whole batches. Scale appears once, on the last line.

Checkpoint cost. Recording the *partial* a trigger folded in is sound, because by Equation (1) a base state plus every later partial is the state. A delta need not beat a snapshot: a stream touching every group each trigger produces a delta the size of the state. Batcher writes a delta only when it is at most half the state and the deltas since the last base, this one included, sum to at most the size of the current state S . Otherwise it writes a new base and resets the sum. A configured length also caps the chain. Between two bases at most $2S$ bytes are written (a base plus deltas summing to at most S), against one full snapshot per trigger for the snapshot-only scheme, and a restore reads one base and at most the configured number of deltas. A test pins the adversarial case: constant state, with half the groups updated on each of ten triggers.

Eviction and late data. A delta chain must also express removal, or replay would resurrect a closed window. The unwatermarked aggregate never evicts. The windowed aggregate removes a prefix of a totally ordered event-time axis, so each delta records the eviction bound it was taken under, and a restore re-applies the largest one after combining. The watermark is the minimum over active partitions of their maximum event time. An idleness timeout (60 s by default) keeps a silent partition from holding it back. A record arriving below the watermark, even from a partition that resumes after timing out, is dropped and counted; windows already emitted do not change. There is no side output.

4 Programming model

SQL is restrictive and so optimizable; a user-defined function is expressive and so opaque. Batcher answers this old bargain three ways, in order of preference. The first removes the callback. `col("x") * col("y"),`
`.str.contains(...),` `.image.to_tensor(...)`

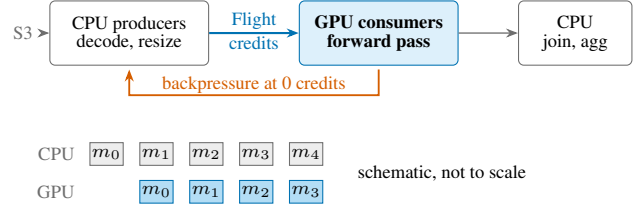


Figure 5: One plan, two resource pools. CPU producers decode and preprocess while GPU consumers run the model. Batches cross between them on the shuffle’s credit-bounded Flight transport [45], so a slow consumer applies backpressure. The timeline sketches the intended overlap (decode of m_{k+1} during the forward pass of m_k) and is not a trace (Section 8.10 measures the effect). Batcher splits at one resource boundary per plan, so the third stage of a CPU→GPU→CPU chain runs in the consumer pool.

and `.audio.mel_spectrogram(...)` all lower to the same `bc_expr: Expr` the SQL front end produces, so each is optimized and executed in Rust, and the JIT may compile it. This lets the filter in Figure 4 sink below the decode. Where a callback is needed, Batcher makes it batch-first. A UDF takes and returns a whole Arrow `RecordBatch`, and a class-based UDF keeps a loaded model across batches, which warm GPU pools depend on. SQL stays, and parses to the same `LogicalPlan`.

“Python never touches a row” therefore holds on the relational and expression path only. A user callback is Python by construction and a model framework runs its own code; Batcher keeps both batch-sized. A `Dataset` is lazy and immutable (each operation returns a new one), and nothing runs before a terminal call, so the program cannot know how many machines will run it. `distributed="auto"` decides at that call. It declines a cluster for an 80,000-row filter whose dispatch would cost more than the query, and takes the staged distributed path for any plan whose shape needs it.

5 Models as operators

For a model in a pipeline, visibility beats feature count: Batcher makes decode and inference expressions, which the optimizer sees alongside the relational operators around them. Image decode, crop, resize and tensor normalization run as Arrow kernels in the data plane, as do audio decode and resampling along with the mel-spectrogram and MFCC transforms. Vector distances and embedding pooling are expressions too. Over the same corpus Batcher decodes 6.4 to $6.6\times$ faster than Ray Data’s `read_images`. The paths also differ in decoder library, thread count and resize semantics, which the experiment does not separate out.

The inference contract. A model call carries obligations a relational operator does not. An expression that references a model binds the model’s weights into its identity, along with the tokenizer and preprocessing, so a new checkpoint is a new expression and never a silently changed answer. Precision is bound in too. By default Batcher runs inference under FP16 autocast on devices that support it, a numerical choice any comparison must

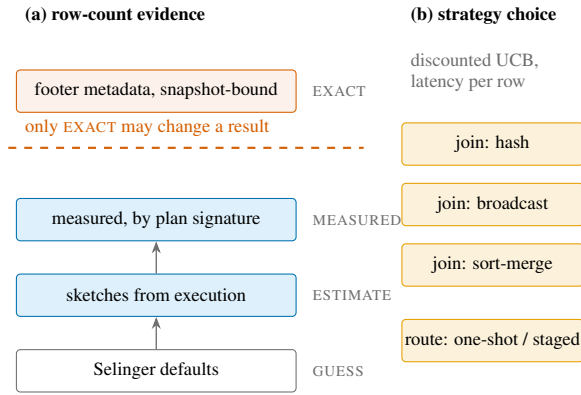


Figure 6: Kyber makes two kinds of learned decision. (a) Row-count evidence, by precedence for estimates: a measured count for the same plan signature beats a sketch, and a sketch beats a default. None of these may change a result. Two rewrites can, dropping a `DISTINCT` over a provably unique key and replacing a provably empty subtree, and they read only exact footer evidence bound to the snapshot of the source they rewrite. (b) Strategy choice, kept apart from estimation. A bandit [5] picks among result-equivalent join algorithms and between one-shot and staged distributed execution.

match or disclose. Batch size is *not* fixed. The engine derives a default from row width and the device’s free memory, splits the tail, and halves any batch that raises a CUDA out-of-memory error. That is legal only for models whose per-row output ignores the other rows in the batch. Models run in evaluation mode under `inference_mode`, which removes the dropout and batch-statistics cases. A padded sequence model must mask its own padding; the engine does not check that it does. The residual batch dependence is numerical: another batch size can select other kernels and move low-order bits under FP16. Batcher therefore promises agreement of predictions across batch sizes (labels, argmax, generated tokens under greedy decoding) rather than bit-equal logits. Stochastic generation falls outside the contract.

One scheduler across CPU and GPU. Run a chain of unlike costs inside one worker and the accelerator idles through every decode. Users usually hand-tune a per-stage CPU-to-GPU worker ratio. In Batcher the engine owns that scheduling problem (Figure 5). Each pool runs its sub-plan through the ordinary executor, so the split moves placement and leaves semantics alone. Ray Data’s streaming batch model [31] attacks the same problem. Its unit of execution is the partition, which buys elasticity and partition-level fault tolerance that Batcher lacks. Batcher instead keeps both stages in one plan, under one cost model with the joins around them.

6 Measurement-driven optimization

An engine that cannot price its data before running must invent the numbers, as a static cost model does, or measure itself and plan from what it saw. Batcher measures (Figure 6), and one rule keeps that safe: every learned value selects among result-equivalent alternatives. Each bandit arm computes the

same relation. A threshold chooses between algorithms that compute the same function, and a partition count only shards. Learning can move how much memory a query reserves, when it spills, how wide a batch is and which equivalent algorithm runs. It cannot move the answer, and a test holds that a cold store reproduces the untuned engine’s results. Learned optimizers that predict plans [27, 32, 33] are not unsafe either, since a validated optimizer steered by hints keeps its semantics. Batcher bounds a bad model’s damage by construction instead, at the price of a weaker signal.

Stores and their keys. Three stores feed planning. *Row counts* are filed under the plan signature of the operator they measure, a hash over the plan’s structure with literals normalized, to which a scan contributes its source identity. For a lakehouse table that identity includes the snapshot or version. For plain files, the learned count carries the files’ size-and-modification digest and is ignored once the digest stops matching. When a query’s root is a projection over an aggregate, its count goes under the aggregate, the node the estimator corrects. Counts update by an exponentially weighted mean and enter estimates as a clipped, shrunk correction to the model’s own estimate. *Cost coefficients* (work units per row for scan, filter, hash build and probe, sort and the rest) are refit from per-operator timings every 64 new observations, shrunk toward their defaults with weight $n/(n+20)$ and clamped within $10\times$ of them. The model’s objective weights CPU work, I/O bytes and network bytes, counting network double. A hardware fingerprint (CPU model and count, SIMD width, cache sizes, memory size class, disk class, GPU) keys the coefficients and bandit state, so a measurement from one machine never prices a plan on another. Row counts skip it, being independent of the machine. *Strategy statistics* feed a bandit over two decisions: the join algorithm per join signature (hash, broadcast, sort-merge) and the distributed route per plan (one-shot or staged). The bandit minimizes latency per million input rows with a discounted, variance-scaled lower-confidence rule. Until an arm has three discounted observations it defers to the cost model. Ties break by arm name, and the first observation of a route arm is dropped as a cold-start outlier.

Three timescales. Two mechanisms run inside an operator at microsecond grain, each standing in for a number a static model would invent. A conjunctive filter orders its infallible predicates by measured time per row removed. On a shape built with two equal-cost predicates, only one of them selective, that is worth $1.86\times$. A parallel aggregate samples the first $\max(4, m/16)$ of its m morsels. If grouping reduced them below 20%, it folds partials per morsel. Otherwise it pre-partitions by key, the third panel of Figure 3 run across cores, and gains $3.7\times$ when grouping reduces nothing. Because the sample is the first morsels, an input with unrepresentative early rows can pick the slower strategy; both strategies return the same relation. At a pipeline breaker the engine re-plans the remainder on real cardinalities. Spark AQE [46] works at the same mechanism and grain, but Batcher runs it on one node as well as on a cluster. It engages only on a plan with a join whose stages each clear 5M rows or about 320 MB, or on any distributed plan whose

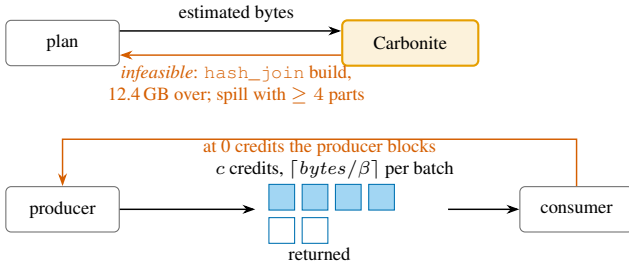


Figure 7: Carbonite’s two interfaces. Above, admission returns the binding constraint and the operator that binds, and its suggested bound sets the spill partition count. Below, credit-based flow control charges by bytes in units of the morsel cap β , so one oversized row cannot slip past the window as a single slot.

shape needs staging. Across executions the engine accumulates the three stores above, and Section 8.7 measures what they buy.

A measurement buys less than the word suggests. Applying a sampled reduction ratio to unprocessed morsels is an inference, and so is applying a fitted crossover to an unseen size. Safety rests on result invariance alone, whatever “measured” seems to promise.

7 Runtime

Three structural rules carry the runtime. Each exists because breaking it is silent. *The subsystems may not import one another.* An optimizer pass that collects its own telemetry compiles and passes every test while corrupting the loop Section 6 depends on. A lint contract blocks that category of coupling, though not every instance. *One expression type serves every CPU tier*, and the crate graph points one way, so no new backend can introduce a second definition of what a plan node means (Section 2 gives the JIT and cuDF checks). Device values are checked only by an opt-in shadow run on the CPU engine, which needs real hardware.

Carbonite, the resource manager. Carbonite decides whether a plan fits and when a query goes out of core, reserves memory, and hands out shuffle credits. It neither rewrites nor computes (Figure 7). A yes-or-no admission answer gives a planner nothing smaller to try, so Carbonite’s verdict names the binding constraint, the operator that binds and suggested bounds. Computed once per query, it does not loop. A memory-infeasible plan with a spill path goes to the spilling executor, its partition count floored at the suggested bound. Otherwise an infeasible plan raises a `PlanError` naming the operator. Flow control uses credits (Section 3.6). Tearing down a consumer’s stream releases a producer blocked on its credits. A consumer that hears nothing for 60 seconds abandons the fetch and enters the recovery path below.

Core, the executor. Core runs the plan it is given and reports what happened. It rewrites no plan, though it makes two operator-local choices between result-equivalent strategies. A conjunctive filter orders its infallible predicates by measured cost per row removed. A parallel aggregate keeps a hash table per morsel

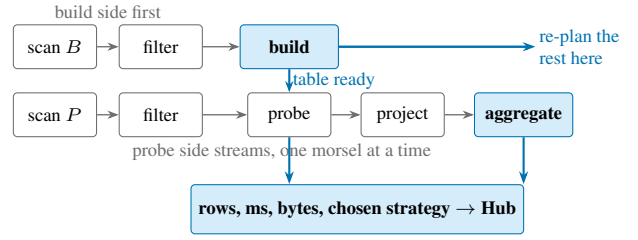


Figure 8: Core’s execution model for a join feeding an aggregate. The build input runs to completion before the probe input streams through the probe, the projection and the aggregate, one morsel at a time. The finished build gives the engine its first exact cardinality, and the stage-boundary re-plan happens there. Every operator reports rows, time, bytes and its chosen strategy to the Metadata Hub.

or pre-partitions by key, depending on the reduction a sample of morsels achieved. Each operator records its chosen strategy beside its row counts and timings, so every Metadata Hub entry carries the decision it was taken under (Figure 8).

7.1 Recovery is a protocol, not a corollary of the algebra

Associativity and commutativity let valid partials be reordered. They say nothing about duplicating or dropping one, and `SUM` and `COUNT` are not idempotent, so worker loss and retry need a protocol above the algebra. Batchter’s has four parts.

Identity. A published bucket is named by $(query, stage, source, destination, epoch)$, where the query identifier is a random 63-bit value per query and the source is the map partition’s index within the stage. For the aggregate reduce, the driver allocates epochs per source. A recompute republishes at the next epoch and reducers fetch exactly the epoch the driver names, so a late publication from an abandoned attempt sits in the store unread. The join, sort and window shuffles reach the same effect differently: they retire a lost source’s replicas, then relocate it to a new worker address.

Completeness. Every map publication in the aggregate, join, sort, window and global window shuffles returns the row count of each bucket it wrote, and the driver passes the counts to the reducers along with the addresses. On every delivery path a reducer compares, per source, what it received with what was published. A shortfall (a missing ticket, a truncated stream, a replica copied from a missing primary) falls over to a replica, then to the recompute path a dead worker takes. Fetching a ticket the peer does not hold raises a retryable error rather than yielding an empty bucket. Only a bucket published empty is expected empty. A short relation therefore fails the stage or gets recomputed, and is never returned. Interior levels of the combiner tree fail closed on a missing partial but carry no counts.

Publication. Replication copies each published bucket to a survivor. It acknowledges only once every ticket in the set is republished with its count verified, and the driver advertises replicas only after every stage has acknowledged. Recovery retires a source’s replicas before republishing it.

Interior of the combiner tree. A tree-reduce builds its interior fallbacks inside one call. If an interior node fails, the attempt raises, a ping finds the dead workers, lost leaves are recomputed

and the whole tree is rebuilt. Interior outputs are named by tree position and map to the same logical chunk across attempts, so a zombie’s late publication carries the same rows as the replacement’s. The tree does not yet count-check a partial the zombie only half-wrote. It relies on the publish being atomic.

Replayability. Recomputing a lost split assumes a source immutable for the query and deterministic operators. File sources are read at the snapshot the plan bound, lakehouse tables at a snapshot identifier. User callbacks are assumed pure. One with external side effects can run twice under recovery, and Batcher does not detect it.

8 Evaluation

Table 3 gives each experiment tag’s hardware, date, lineup, repetition and statistic. No timing is reported for a query whose result disagrees with DuckDB under Section 3.5 (floats within 1.5×10^{-6} absolute, 10^{-9} relative). Model workloads are gated on prediction agreement. Every engine is measured up to a materialized `pyarrow.Table`.

8.1 Do the schedulers agree?

Beyond the property test of Equation (1), an operator matrix crosses every execution mode (collect, spill, streamed batches, distributed) with every value edge (nulls, empty input, one row, duplicates, -0.0 and NaN , descending order). The defects that reached users were each a non-default flag on a non-default path that every single-axis test passed: a descending sort returned unsorted data under spill, and a distributed `GROUP BY` on a float key split a group in two.

Alone, `distributed=True` runs one worker, which computes what one node computes. E3 therefore compares `distributed=False` with two workers on TPC-H `sf1` over Parquet split eight ways, since an in-memory table cannot be divided and would keep the distributed side on one worker. A control known to diverge, `LIMIT 3` over an unordered `group_by`, must return three rows of the full aggregate but may pick different ones. It moved: single-node kept groups $\{3, 7, 10\}$, two workers $\{3, 5, 12\}$. Against that, all 22 queries returned equivalent relations, each on two workers over two partitions.

At four workers the run first failed q10 and q13 with a truncated shuffle file, rewritten in place by a straggler’s backup task while a reducer read it. Shuffle files now go to a private temporary and are renamed into place, after which 32 repeats of q3, q10, q13 and q18 at four workers agreed. The earlier version of this experiment ran one worker on unsplittable tables, which is why it never saw the defect.

8.2 Single-node relational

Table 4 has Batcher leading DuckDB on all six E1 suites over identical Arrow, from 0.83 on H2O `group_by` to 0.16 on ClickBench. Against the compressed native store it leads five and ties on H2O `group_by` (1.05 here, 0.95 three days later

on a loaded box, both inside the spread of Table 3). E12 repeats two suites five times for intervals. On TPC-H `sf1` Batcher measured 0.86 against DuckDB-native (range 0.75 to 0.88), 0.30 against DuckDB over Arrow and 0.55 against Polars; on H2O `group_by`, 1.03 against DuckDB-native (1.00 to 1.04), 0.79 over Arrow and 0.49 against Polars. Every engine passed every case in every repeat.

That run qualifies Table 4 twice. Its native-store TPC-H ratio, 0.86 against E1’s 0.72, shows one sweep on a loaded machine moving by more than the quiet-box spread. And the geometric mean disagrees with the sum: the median TPC-H suite *sum* was 715 ms for Batcher and 716 ms for DuckDB-native, a tie, because Batcher wins most short queries and loses long ones such as q21 (108 against 72 ms).

The lower block depends on conditions. TPC-H `sf10` at 0.963 reads in-memory Arrow on the 96-core box. From Parquet files on a loaded 48-core box (2026-09-16) the same suite measured 1.52, losing 21 of 22 queries, and that is what a user with files on disk gets. The single-node crossover with DuckDB thus lies between `sf1` and `sf10`, depending on how data arrives. TPC-DS completed all 99 queries at 0.98 with every check passing. A later run on a loaded box read 0.99 to 1.40 across two passes, a spread due to load. Batcher loses the Join Order Benchmark, whose correlated predicates defeat independence assumptions [30]: its total time beats DuckDB’s (8.1 s against 8.9) but its geometric mean is 11% worse, a loss spread over many short queries.

Margins against the rest of the field are wider (separate runs, 2026-08-28, 92 cores). Daft [18] measured 0.21 on TPC-H `sf1`, 0.17 on `sf10`, 0.11 on ClickBench and 0.07 on the operator mix; PyArrow 0.03 on the operator mix, the only suite it can express; local-mode Spark 20 to $50\times$ behind on TPC-H `sf1`. Spark is not built for local mode, and no claim rests on that number.

8.3 What is storage and what is execution

Moving DuckDB from Arrow views to native tables changes its physical representation, planner statistics, scan and possibly plan, so the two DuckDB columns of Table 4 compare two *configurations* and do not separate storage from execution. The configuration does split the 40 cases where some engine beat Batcher. In 19, Batcher beats DuckDB over the same Arrow and loses only to DuckDB-native. In the other 21, DuckDB over Arrow or Polars is faster too, so Batcher’s execution loses.

One mechanism shows in both. Batcher’s string group-by costs 1.15 to $1.22\times$ its integer group-by, while DuckDB groups low-cardinality strings faster than integers, as grouping on dictionary codes does. Batcher has a code-grouping kernel no query can reach: its Parquet reader decodes dictionaries during the read, and its FFI boundary rewrites a surviving dictionary to its value type. Arrow specifies dictionary arrays and a view layout for strings; Batcher chose to use neither in its kernels. Closing the gap needs the reader, the boundary and a logical-versus-physical type split. A boundary-only version regressed plain projections to 0.63 and was reverted. Zone maps are the other mechanism. On a six-million-row table, `WHERE l_comment LIKE 'zzzzq%'` costs DuckDB-native 0.49 ms, because block statistics prove no match, and

Tag	Experiment	Hardware	Date	Lineup, isolation, repetition	Statistic
E1	six single-node suites (Table 4, top)	48-core (24 + SMT), 92 GiB, shared	2026-09-13	4 engines, one process per case, 5 runs	min per case; geomean
E2	TPC-H sf10, JOB; TPC-DS sf1 (Table 4, bottom)	96-core, 184 GiB; TPC-DS on E1's box	08-25; 09-11	2 engines, in-process, 5 runs	min per case; geomean
E3	scheduler equivalence (Section 8.1)	E1's box, local Ray	2026-09-23	distributed=False vs. 2 workers	agree / differ
E4	CPU cost, TPC-H sf1	96-core, 184 GiB	2026-08-18	2 engines, in-process, 2 rounds	getrusage; geomean
E5	cold small queries	96-core, load ≈ 18	2026-07-29	fresh literal per run	median of 10
E6	streaming backlog drain	96-core; batch arm on E1's box	08-18; 09-23	per engine, 5 runs	median
E7	TPC-H sf100 pipelines (Table 5)	1 head (no worker CPU) + 100 $\times$ (4 CPU, 8 GB)	2026-09-09	engine-major, best of 2, 400 s timeout	min
E8	worker loss (Table 6)	8 workers, one box, local Ray	2026-08-18	3 rounds per arm	min
E9	image scoring, 100k JPEGs	6 $\times$ g4dn.2xlarge (T4)	2026-09-06	engine-major, 1 run	wall
E10	GPU workloads (Table 7)	8 $\times$ T4 nodes + 8 CPU nodes	Jul.-09-09	Batcher; Ray Data where named	wall, NVML
E11	learning vs. plan-cache ablation (Figure 9)	E1's box, shared	2026-09-23	fresh process per arm, 8 runs, 4 rounds	geomean vs. own run 8
E12	repeated TPC-H sf1 and H2O group_by	E1's box, shared	2026-09-23	4 engines, one process per case, best of 5, 5 repeats	median and range of geomeans

Table 3: Experiment protocol; every number in the paper carries one of these tags. “Shared”: other work ran on the machine. On E1’s box, `--repeat 5` gave a suite-geomean spread of 1.4 to 2.3% (11% for a single case), so suite ratios a few percent apart do not differ.

Suite	<i>n</i>	DuckDB			
		Arrow	native	Polars[49]	slower
JSON	5	0.32	0.35	0.01	0/5
H2O.ai [22] join	5	0.58	0.63	0.51	0/5
ClickBench [13]*	43	0.16	0.65	0.37	15/43
TPC-H [48] sf1	22	0.25	0.72	0.54	6/22
Operator mix*	46	0.47	0.75	0.16	13/46
H2O.ai group_by	10	0.83	1.05	0.53	6/10
<i>E2: larger data and harder planning; separate runs</i>					
TPC-H sf10, in-memory	22	—	0.963	—	—
TPC-DS [47] sf1	99	—	0.98	—	—
Join Order [30]	113	—	1.11	—	—

Table 4: Single-node relational results, b/x ; below 1.00 means Batcher is faster. Top: E1, every case correctness-gated for every engine; the E1 logs keep suite ratios but not per-engine failed-case counts, which E12 records. The last column counts cases where any engine beat Batcher. Bottom: E2, comparable within a row only. *Includes five ClickBench queries and two operator-mix cases answered from a recorded statistic without executing (Section 8.4).

Batcher 7.95 ms reading the column (measured apart from the 40-case split).

8.4 Memoized answers, separated from executed ones

In E1, Batcher answers five ClickBench queries and two operator-mix cases from recorded statistics without executing them, a feature that says nothing about kernels. On the one run that recorded the split (2026-08-15, against DuckDB-native), excluding them moved ClickBench from 0.63 to 0.77 over 38 queries and the operator mix from 0.64 to 0.76 over 17 cases. The E1 table includes and marks them. Contamination also runs the other way. Every engine returns a materialized table, and on a query returning nine million rows that export is about half of DuckDB’s measured time. Users pay it, hence the boundary, but it says nothing about DuckDB’s execution.

8.5 Compute cost, not only latency

An engine can buy latency with compute. Batcher and DuckDB both execute in-process, so `getrusage` charges both engines’ threads to one counter. On TPC-H sf1 (E4), excluding q15, which Batcher answers from metadata, the geometric means are $0.78\times$ DuckDB’s wall time for $1.86\times$ its CPU time (21.9 effective cores against 9.2); over all 22 queries, 0.74 and 1.47. Batcher thus buys a $1.3\times$ latency win with about twice the CPU. That measures efficiency and prices nothing: a rented machine is not charged by process CPU utilization, and turning CPU-seconds into cost needs an allocation model this paper does not supply.

Cold and small queries. A best-of- N suite never plans a query for the first time. E5 uses a fresh literal per run, which defeats the plan cache while the process and query structure stay warm. Optimizing a filter $\rightarrow$ join $\rightarrow$ group-by then costs 2.75 ms (median of ten). End to end over twelve such shapes at $n = 10^4$, Batcher runs at $0.80\times$ DuckDB on a filter and $0.27\times$ on a count (*). The narrowest shape still loses warm, 4.07 ms against 2.04, of which planning is 0.002 ms. The rest sits elsewhere in the per-query path, the FFI crossing included. A cold *process* and a new plan *structure* would each be separate measurements.

8.6 Streaming throughput and driver cost

Draining a four-million-row Parquet backlog through a grouped aggregation over 1,000 keys, correctness-gated (E6), Batcher’s micro-batch path runs at 18.9M rows/s against Spark Structured Streaming’s [3] 6.0M. DuckDB answers the same aggregation as one batch query in 15.9 ms, $13\times$ faster than Batcher’s streaming path, but that comparison mixes engines. Within Batcher, over the identical 4M-row, 16-file backlog ending in the same 1,000-row Arrow table, the batch path took a median 18.5 ms and the micro-batch path 50.1 ms: a $2.7\times$ driver cost (E6, 2026-09-23, load 0.5 per core; $2.4\times$ in a second run at 3.2). DuckDB took 11.6 ms in that run. Most of the gap to DuckDB’s batch query is thus the micro-batch driver, which plans each micro-batch and, per trigger, logs offsets and writes a checkpoint. Operator and

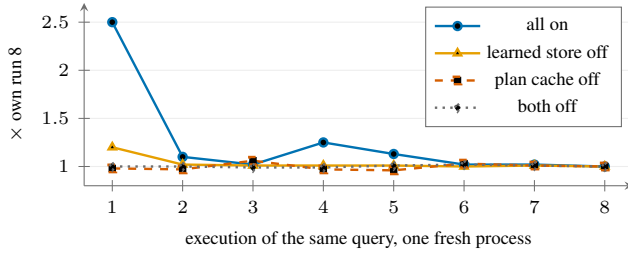


Figure 9: E11: four-arm ablation, TPC-H sf1, round r4 (load 0.3 per core); geometric mean over 22 queries of each execution’s time over the same arm’s eighth. Each curve is normalized to itself, so it shows what repetition gains, not which arm is faster; the learned-store-off arms are about 4 to 10 $\times$ slower in absolute terms because they recompute column statistics.

planning cost make up the rest. A finite backlog amortizes the source lag, uneven partitions and offset recovery that a steady arrival rate would expose, and we have not run that experiment.

8.7 Cross-execution adaptation

A second execution is faster for many reasons, so a before-and-after comparison cannot establish a learning loop. E11 runs each TPC-H query eight times in a fresh process under four configurations (everything on, plan cache off, learned store off, both off), normalizing each curve to its own steady state. Figure 9 shows the round that started on the quietest box, and three other rounds agree on its shape. With everything on, the first execution costs 2.50 $\times$ the steady state (2.64 and 2.77 in two other rounds); with the plan cache off, 0.98. *The plan cache accounts for the whole first-to-steady improvement on repeated TPC-H queries*, and nothing here shows measured row counts making a later execution faster. With the learned store off and the plan cache on, the curve is 1.2 $\times$, but that arm also stops caching per-column statistics across queries, which alone raises steady-state time from about 16 ms to about 150 ms a query. The store’s measurable value on this workload is therefore statistics reuse. The experiment cannot separate execution feedback from it, and a switch that isolates the feedback path is the next measurement. These runs predate the fix that files an aggregate’s measured count where the estimator reads it (Section 6).

8.8 Scale-out

At sf100 (E7), Batcher leads Daft on every pipeline that produced a number, and Ray Data did not finish three of the four inside the timeout. Engines ran in a fixed, unrandomized order on shared object storage, so a later engine could see a warmer bucket. An earlier round put Batcher behind, with every cause in the control plane and none in an operator: a derived worker count read as a user override suppressed the cluster fill, any stage containing a UDF fell back to the single-node orchestrator, and a join reducer sent 3.75M rows back through Python. On a separate 16-node fleet of 8-CPU workers reading TPC-H from S3 (2026-07-12), Batcher led Daft 1.72 to 2.23 $\times$ on join across sf1, sf10 and sf100 and trailed it at 0.84 $\times$ on the storage-bound `filter_count` at sf100. Two fleet sizes on different

Pipeline	Batcher	Ray Data	Daft	vs Ray	vs Daft
filter, count	282 ms	2,902 ms	1,701 ms	10.3 $\times$	6.0 $\times$
group_by	322 ms	t/o	1,200 ms	—	3.7 $\times$
join	1,845 ms	t/o	201,165 ms	—	109 $\times$
UDF map	744 ms	t/o	err	—	—

Table 5: E7: TPC-H sf100 Parquet on S3; 100 four-CPU workers plus a head with no worker CPU; engines run sequentially (Batcher, Ray Data, Daft), best of two, 400 s timeout; result signatures compared across engines before a timing is kept. t/o (timeout) and err (engine error) get no ratio and mean the engine did not finish this experiment, not that it cannot express the pipeline. Batcher’s mean fleet CPU utilization ranged 17% to 60% by pipeline. Excluded: `scan_count`, which Batcher answers from Parquet footers in 2 ms, a better plan rather than distributed throughput.

Arm	ms	vs r=1	vs own	Result
no fault, r=1	258.0	1.00 $\times$	1.00 $\times$	equivalent
no fault, r=2	172.7	0.67 $\times$	1.00 $\times$	equivalent
1 lost at reduce, r=1	733.6	2.84 $\times$	2.84 $\times$	equivalent
1 lost at reduce, r=2	2167.1	8.40 $\times$	12.55 $\times$	equivalent
2 lost at reduce, r=2	2215.4	8.59 $\times$	12.83 $\times$	equivalent
1 lost during map	1723.1	6.68 $\times$	6.68 $\times$	equivalent
2 lost during map	1755.7	6.81 $\times$	6.81 $\times$	equivalent

Table 6: E8: five injected worker-loss arms and two no-fault controls on a grouped aggregate over 8M rows and 5,000 keys, 8 workers on one machine; r is the shuffle replication factor. A worker lost *at reduce* has already published its buckets, which vanish with it; one lost *during map* has published nothing, and its split is relocated. *vs r=1* is the cost against the default configuration, *vs own* the cost to a deployment already running with replicas.

workloads do not make a scaling curve, and Batcher has not measured one.

8.9 Worker loss

All five fault arms return a relation equivalent to the single-node answer (Table 6), which is the purpose of Equation (1) and the protocol of Section 7.1, here tested against killed workers instead of generated partitionings. Losing two workers cost about what losing one did, in both phases. Independent partials permit parallel regeneration, but the totals cannot show it: a fixed detection timeout would produce the same shape, and E8 recorded no per-phase trace. Replication did not pay. Copying every published bucket to a survivor turned a 2.84 $\times$ recompute into an 8.40 $\times$ recovery. Replication assumes that re-reading a source from object storage is expensive, which is false on one machine with an in-memory source, so this run measures its fixed cost and not its benefit.

8.10 GPU and model pipelines

Table 7 reports coverage only, across images, video, audio, text and generation. The zero-configuration row carries the interface claim. With no batch size, `ds.map_batches(Model, num_gpus=1)` ran at 2,451 images/s against 2,504 with `batch_size=128`, 2.1% slower, because the engine derives a default from row width and free device memory, overlaps decode with the forward pass, and halves a batch on an out-of-memory

Workload	Model (weights)	Input and batch size	Rate	GPU busy	Unit
Image classification, fused	ResNet-50 (random init)	131,072 JPEGs; 128	2,576	78%	images/s
Decode + infer, two stages	ResNet-50 (random init)	131,072 JPEGs; 128	2,504	81%	images/s
Image embeddings, 2048-d	ResNet-50 features (random init)	12,000 images; 256	2,502	80%	images/s
Zero-configuration	ResNet-50 (random init)	131,072 JPEGs; engine default	2,451	82%	images/s
Two models per GPU	EfficientNet-B0 (random init)	49,000 images; num_gpus=0.5	6,764	89%	images/s
Video clips, per-frame	ResNet-18 (random init)	4,096 16-frame clips; 64	2,075	—	clips/s
Audio features	mel on CPU, ResNet-18	16,384 clips; 128	38,546	—	clips/s
Text embeddings, 384-d	all-MiniLM-L6-v2 (pretrained)	8,192 texts; 256	33,611	—	texts/s
LLM generation	gpt2 FP16 (pretrained), greedy	2,048 prompts; 32; 32 new tokens	815	—	prompts/s
Image generation	DDPM CIFAR-10 (pretrained), 20 DDIM steps	2,048 images; 64	169	—	images/s
Training ingest	none (iter_torch_batches)	200,000 rows, 1024-d; 256	1.06M	—	rows/s

Table 7: E10: eleven GPU workloads on 8×T4, each an operator in an ordinary plan gated on agreement with a single-process reference; seven distinct models, the ResNet-50 rows sharing one network. All rows but zero-configuration pass the batch size shown. *GPU busy* is NVML’s utilization counter (fraction of time a kernel was executing), sampled every 0.25 s per device, averaged, and recorded on five rows only; it says nothing about occupancy or achieved FLOPs. Torchvision networks are randomly initialized under a fixed seed: the same kernels as trained weights, but prediction agreement then checks determinism, not accuracy.

error. That is one model at one row width, and Batcher has not published a batch-size sweep against it.

Head-to-head, three results hold and two go the other way. All ran Batcher under its default FP16 autocast and the comparators at their default precision (FP32 for the torch workloads), so they compare defaults, not equal numerical work.

Image scoring (E9): six single-T4 nodes, 100,000 small JPEGs on S3, and a dense-matrix scorer in cupy that every engine built identically, not a trained vision model. Batcher finished in 18.72 s against Ray Data’s 44.19 and Daft’s 101.10 (2.36× and 5.40×), with checksums agreeing within 0.15%, the JPEG decoders’ difference. The scorer is 15% of Batcher’s time, so the margin lies in object reads and decode, and in scheduling.

LLM generation (E10): gpt2, greedy decoding, 32 new tokens per prompt, 65,536 prompts of 10 to 20 tokens. Batcher produced identical text at 1,521 prompts/s against Ray Data’s 227, 6.71×. At 32,768 prompts the ratio was 13.4×, but Ray Data’s rate was still rising with scale, so the larger run is the better estimate and a larger one might read lower again. Scale cut the text-embedding ratio likewise, from 29.5× at 8,192 texts to 1.52× at 1.3 million, where Batcher’s rate was flat and Ray Data’s had caught up. A ratio taken while the slower engine is still amortizing its start-up measures the start-up.

Compute-bound inference crosses over with size: 8.6× at 125,000 rows, 2.5× at 500,000, 0.96× at two million, 0.76× at four. The cause is that a fused CPU-then-GPU map runs Batcher’s CPU stage in threads inside the device-holding actors and Ray Data’s in processes. A factor there means nothing without its row count.

Two-stage decode-then-infer over 131,072 JPEGs goes the other way: 0.85× Ray Data (2,007 against 2,373 images/s, identical predictions). Ray Data’s devices are 80% busy at 297 images/s per T4, near the 285 the same T4s reach fused; Batcher’s are 73% busy, and the loss is device idle time in the hand-off. Sweeping decode concurrency and the credit window moved it under 3%. Double-buffering the terminal consumer reached 2,232 images/s at 80% busy but was reverted, because the autoscaler read the emptied queue as over-provisioning and halved the worker pool on the one-stage benchmark.

29 GB batch inference over 200,000 images measured 1.51× Ray Data once. Later runs, after cluster nodes had begun drop-

ping out under sustained load, gave Batcher 989 to 1,666 images/s against Ray Data’s steady 1,506 to 1,621. We report it as unreproduced. An earlier 0.42× came from two read-path defects, a `uint8` tensor widened to `int64` at the FFI boundary and the row-count read batch of Section 3.6. Fixing both cut a single-node reproduction’s peak RSS from 11.47 GB to 3.09 GB.

9 Limitations and threats to validity

The experiments span several workload classes and establish no uniform superiority. Table 8 gives the support matrix with evidence per cell; Appendix B separates what is evaluated here from what is only implemented.

Storage is the largest open gap. Batcher owns no storage format and decodes dictionaries at the leaf, which costs it the string and zone-map cases of Section 8.3 and the sf10-from-Parquet result of Section 8.2. The fix lies in the reader, the boundary and the type system, not in a kernel.

Streaming is micro-batch and falls short of Flink’s guarantees. Batcher has no persistent operator graph, barrier protocol or keyed state backend with point lookups, and its driver is a single point of failure with no standby. Exactly-once output holds for the sinks of Section 3.7 under a single writer. Two concurrent queries writing one table are not coordinated. A worker lost mid-trigger makes the driver restart the trigger from its checkpoint. Unit tests inject failures on that path and crash-and-restart tests interrupt every point of the commit sequence, but no test yet kills a real worker during a distributed micro-batch.

Distributed coverage has holes. An inequality join scales only by broadcasting its right side. Several global window functions refuse to distribute, and a plan gets one CPU-to-GPU resource boundary. Recovery was tested only with a surviving driver and on one machine, and complete pipelines have no node-count scaling curve (Sections 8.8 and 8.9).

Continuous integration checks the device tier by type, not by value. CI installs neither Ray nor a GPU. The distributed and GPU results rest on recorded runs at the revisions Table 3 dates, and until one is repeated, later changes on those paths are covered only by the schema contract and the pandas-backed translation tests.

Threats to validity. *Best of N, warm:* E1 reports the minimum of five runs of the same query, a shape a memoizing engine wins, and Batcher’s session carries a plan cache and learned store across those repeats where DuckDB and Polars have neither. E11 puts the plan cache at about $2.5\times$ on a repeated TPC-H query, so a warm best-of-five favours Batcher. E12 gives the spread, not a cold comparison. *Correctness gate:* a case Batcher fails leaves its geometric mean, which can flatter the aggregate, so E12 reports requested, correct and timed counts per engine. *Timed path:* the gate precedes the timed repetitions, between which adaptive strategies can change. Each strategy is result-equivalent by construction and separately covered by the oracle tests, but the timed physical plan is not re-checked. *Precision:* the GPU head-to-heads pit Batcher’s FP16 default against the torch comparators’ FP32. *Comparator configuration:* we found and fixed an adapter defect that left Ray Data with one of 96 cores, three Spark adapter defects, and a Daft startup timeout shorter than the fleet’s first-use environment build. Others may remain.

10 Related work

Vectorized and compiled single-node execution. Columnar batch-at-a-time execution begins with MonetDB/X100 [8], compiled execution with HyPer [37]; Umbra [38] contributed the inline-prefix string representation Section 8.3 names as a gap here. Batcher is morsel-driven [29] and vectorized, with a Cranelift-compiled path [11]; DuckDB [40] is its closest single-node comparator. Velox [39] and DataFusion [28] are embeddable engines that leave distribution to a host. DataFusion shares Rust, Arrow and much of the operator set with Batcher. Its execution-plan trait admits any operator implementing it, so a mergeable form is available but not required. Photon [7] is a vectorized engine inside a lakehouse runtime rather than an embeddable library.

Distributed dataflow and warehouses. The mergeable triple comes from MapReduce’s combiner [16] and Gray et al.’s algebraic aggregates [21]; the contribution here is refusing any stateful operator that declares no contract. Spark [2, 50] runs local and cluster modes through the same physical operators, with binary memory management and whole-stage code generation, and its adaptive execution re-plans at shuffle boundaries [46]. Batcher’s C2 contract follows Volcano’s separation of operators from exchange [20]. Cloud warehouses [15, 43] answer the same questions as a service, not a library, and Batcher’s transactional sinks write the lakehouse formats [4].

Streaming and unified execution. Batcher’s micro-batch path adopts Structured Streaming’s unbounded-table framing [3]. Summingbird [9] made the monoid the unifying abstraction so one program compiles to a batch and an online engine, and the Dataflow model [1] gave the two one specification, whose CombineFn has the shape of C1. Both unify the program above engines; Batcher makes the contract the admission test for one engine’s operators. Flink [12], a single runtime treating

bounded input as a special case, has what Batcher lacks: a persistent operator graph, barrier snapshots and a keyed state backend. Naiad [36] is another single-dataflow answer. DBSP [10] derives incremental operators from a query algebra with signed updates, expressing removals Batcher’s partial log cannot.

Adaptive and learned optimization. Estimation begins with Selinger [42]. Stage-boundary re-planning descends from ed-dies [6] and mid-query re-optimization; Spark AQE is its closest deployed relative at the same grain. Learned optimizers [27, 32, 33] use models to choose which plan runs. Bao steers a validated optimizer with hints instead of replacing it, so it differs from Batcher in the size of the decision a model makes, not in answer correctness. Batcher takes a lighter route, with sketches [14, 19], refit cost coefficients and a bandit [5] over result-equivalent strategies: its cold start is the untuned system, and it gives up correcting a badly wrong plan. Leis et al. [30] supply both the standing argument for measured cardinalities and the benchmark Batcher still loses.

Data systems for machine learning. The model evaluation is measured against Ray [34] and Ray Data [31], the closest system to Section 5. Daft [18] is the closest multimodal relational engine; its documentation describes cost-based join reordering and multimodal-aware scheduling, so a comparison must name a release. We compare relational and inference pipelines against Daft 0.7.24 but have not run a full mixed relational-and-model pipeline on it. Batcher’s computed epoch permutation comes from MosaicML’s StreamingDataset [35]. NoScope [23] and BlazeIt [24] put a model inside the optimizer, approximating with cascades, proxy models and control variates where the semantics allow; for their target queries those techniques are stronger than anything here. Batcher makes decode, embedding and inference ordinary expressions and approximates nothing.

11 Conclusion

The evaluation supports a narrower claim than “one engine wins”. On named single-node suites Batcher is faster than DuckDB over identical Arrow input and competitive with DuckDB’s own storage, using about twice the CPU for its latency. On one 100-worker fleet it finishes TPC-H pipelines Ray Data did not and leads Daft on the rest. It loses the Join Order Benchmark and sf10 read from Parquet, trails Ray Data on a two-stage GPU pipeline, and has a weaker streaming path than a Flink-class system. Its repeated-query speedup comes from a plan cache any engine could add, not from the learning loop.

Batcher tests one design rule: a stateful operator enters the engine only with a declared execution contract, and the scheduler dispatches on nothing else. Three contracts cover the whole vocabulary once the second is read as partition-local evaluation and the third as replicate-or-iterate under a budget, and a registry with an exhaustive test keeps a new operator from skipping the classification. Stating the contracts precisely forced several repairs to the engine, each to code that had passed every test: integer SUM overflow that depended on the partition count, a

shuffle that could return a short relation when a bucket went missing, a checkpoint policy whose write bound was false, a learned-cardinality path that filed every measurement where nothing read it, a transport that could not carry a single batch over 4 MiB, and shuffle files that a straggler’s backup could truncate while a reducer read them. The last surfaced only when the paper’s own equivalence experiment was rerun with a lever that moved. The part of the hypothesis that matters most for the design remains untested: whether the contracts survive richer physical representations (dictionary strings, a keyed state backend) without a fourth contract or a second implementation creeping back in.

References

- [1] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.
- [2] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia. Spark SQL: Relational data processing in Spark. In *ACM SIGMOD International Conference on Management of Data*, 2015.
- [3] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *ACM SIGMOD International Conference on Management of Data*, 2018.
- [4] M. Armbrust, T. Das, S. Paranjpye, et al. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12), 2020.
- [5] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002. doi: 10.1023/A:1013689704352.
- [6] R. Avnur and J. M. Hellerstein. Eddies: Continuously adaptive query processing. In *ACM SIGMOD International Conference on Management of Data*, 2000.
- [7] A. Behm, S. Palkar, U. Agarwal, et al. Photon: A fast query engine for lakehouse systems. In *ACM SIGMOD International Conference on Management of Data*, pages 2326–2339, 2022. doi: 10.1145/3514221.3526054.
- [8] P. A. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-pipelining query execution. In *Conference on Innovative Data Systems Research (CIDR)*, 2005.
- [9] O. Boykin, S. Ritchie, I. O’Connell, and J. Lin. Summingbird: A framework for integrating batch and online MapReduce computations. *Proceedings of the VLDB Endowment*, 7(13), 2014.
- [10] M. Budiu, F. McSherry, L. Ryzhyk, and V. Tannen. DBSP: Automatic incremental view maintenance for rich query languages. *Proceedings of the VLDB Endowment*, 16(7), 2023.
- [11] Bytecode Alliance. Cranelift code generator, 2024. URL <https://cranelift.dev>.
- [12] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4), 2015.
- [13] ClickHouse, Inc. ClickBench: A benchmark for analytical databases, 2024. URL <https://benchmark.clickhouse.com>.
- [14] G. Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005. doi: 10.1016/j.jalgor.2003.12.001.
- [15] B. Dageville, T. Cruanes, M. Zukowski, et al. The Snowflake elastic data warehouse. In *ACM SIGMOD International Conference on Management of Data*, 2016.
- [16] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.
- [17] J. Demmel and H. D. Nguyen. Fast reproducible floating-point summation. In *21st IEEE Symposium on Computer Arithmetic (ARITH)*, pages 163–172, 2013. doi: 10.1109/ARITH.2013.9.
- [18] Eventual Computing. Daft: A distributed query engine for large-scale multimodal data, 2024. URL <https://getdaft.io>.
- [19] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In *Conference on Analysis of Algorithms (AofA)*, 2007. doi: 10.46298/dmcs.3545.
- [20] G. Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.
- [21] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data Mining and Knowledge Discovery*, 1(1):29–53, 1997.
- [22] H2O.ai. db-benchmark: Database-like ops benchmark, 2023. URL <https://github.com/h2oai/db-benchmark>.
- [23] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia. NoScope: Optimizing neural network queries over video at scale. *Proceedings of the VLDB Endowment*, 10(11), 2017.
- [24] D. Kang, P. Bailis, and M. Zaharia. BlazeIt: Optimizing declarative aggregation and limit queries for neural network-based video analytics. *Proceedings of the VLDB Endowment*, 13(4), 2020.
- [25] Z. Karnin, K. Lang, and E. Liberty. Optimal quantile approximation in streams. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2016. doi: 10.1109/FOCS.2016.17.
- [26] Z. Khayyat, W. Lucia, M. Singh, M. Ouzzani, P. Papotti, J.-A. Quiané-Ruiz, N. Tang, and P. Kalnis. Lightning fast and space efficient inequality joins. *Proceedings of the VLDB Endowment*, 8(13), 2015. doi: 10.14778/2831360.2831362.
- [27] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.
- [28] A. Lamb, Y. Shen, D. Heres, J. Chakraborty, M. O. Kabak, L.-C. Hsieh, and C. Sun. Apache Arrow DataFusion: A fast, embeddable, modular analytic query engine. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*, pages 5–17, 2024. doi: 10.1145/3626246.3653368.
- [29] V. Leis, P. Boncz, A. Kemper, and T. Neumann. Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In *ACM SIGMOD International Conference on Management of Data*, 2014.
- [30] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3):204–215, 2015.
- [31] F. S. Luan, R. Y. Wang, Y. Gu, Z. Mao, C. Lin, A. Kamsetty, H. Chen, C. Su, B. Veeramani, S. Lee, S. Cho, C. Zinzow, E. Liang, I. Stoica, and S. Wang. The streaming batch model for efficient and fault-tolerant heterogeneous execution. arXiv:2501.12407, 2025. URL <https://arxiv.org/abs/2501.12407>.
- [32] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. Neo: A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.
- [33] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska. Bao: Making learned query optimization practical. In *ACM SIGMOD International Conference on Management of Data*, 2021.
- [34] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
- [35] MosaicML / Databricks. StreamingDataset: Fast, accurate streaming of training data from cloud storage, 2023. URL <https://github.com/mosaicml/streaming>.
- [36] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi. Naiad: A timely dataflow system. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2013.
- [37] T. Neumann. Efficiently compiling efficient query plans for modern hardware. *Proceedings of the VLDB Endowment*, 4(9):539–550, 2011.
- [38] T. Neumann and M. Freitag. Umbra: A disk-based system with in-memory performance. In *Conference on Innovative Data Systems Research (CIDR)*, 2020.
- [39] P. Pedreira, O. Erling, M. Basmanova, et al. Velox: Meta’s unified execution engine. *Proceedings of the VLDB Endowment*, 15(12):3372–3384, 2022. doi: 10.14778/3554821.3554829.
- [40] M. Raasveldt and H. Mühleisen. DuckDB: An embeddable analytical database. In *ACM SIGMOD International Conference on Management of Data*, 2019. Demonstration.
- [41] D. Reinsel, J. Gantz, and J. Rydning. The digitization of the world: From edge to core. IDC White Paper US44413318, International Data Corporation, November 2018. URL <https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf>.
- [42] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *ACM SIGMOD International Conference on Management of Data*, 1979.
- [43] R. Sethi, M. Traverso, D. Sundstrom, et al. Presto: SQL on everything. In *IEEE International Conference on Data Engineering (ICDE)*, 2019. doi: 10.1109/ICDE.2019.00196.
- [44] The Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory analytics, 2024. URL <https://arrow.apache.org>.
- [45] The Apache Software Foundation. Arrow Flight RPC, 2024. URL <https://arrow.apache.org/docs/format/Flight.html>.
- [46] The Apache Software Foundation. Adaptive query execution — Spark SQL performance tuning guide, 2024. URL <https://spark.apache.org/docs/latest/sql-performance-tuning.html>.
- [47] Transaction Processing Performance Council. TPC Benchmark DS standard specification, 2021. URL <https://www.tpc.org/tpcds/>.
- [48] Transaction Processing Performance Council. TPC Benchmark H standard specification, 2022. URL <https://www.tpc.org/tpch/>.
- [49] R. Vink and Polars contributors. Polars: DataFrames for the new era, 2024. URL <https://polars.rs>.
- [50] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

Property	batch	micro-batch	distributed	evidence
Relation equal under Section 3.5	yes	yes	yes	matrix; E3
Float aggregates bit-identical across modes	no	no	no	reassociation
Float aggregates bit-identical, same mode and partitioning	yes	yes	yes	oracle tests
Integer SUM error independent of partitioning	yes	yes	yes	unit + diff. tests
Unordered LIMIT, tied row_number, array_agg order	open	open	open	subset / rank-set checks
Event-time windows; late rows dropped and counted	n/a	yes	yes	tests
Watermark barriers, retractions, keyed state backend	n/a	no	no	—
Exactly-once: Delta, file, keyed DB	n/a	yes	yes	replay tests
Exactly-once: Iceberg	n/a	yes	refused	replay tests
Exactly-once: unkeyed DB append; driver failover	n/a	no	no	—
Worker-loss recovery, batch shuffle	n/a	n/a	yes	E8; fault tests
Worker-loss recovery, mid-trigger	n/a	restart	restart	injected-failure tests
Inequality join	yes	—	replicate or raise	tests
Recursive CTE (linear)	yes	—	iterate, capped	diff. tests
Spill: agg, distinct, sort, join, window	yes	partial	yes	spill tests
CPU→GPU boundaries	one per plan			E10

Table 8: Support by execution regime, with the evidence for each row. *Open* marks a choice SQL leaves open (Section 3.5); *order_by* closes each. *Partial* spill: the windowed streaming aggregate spills closed windows and the unwatermarked one raises at its state ceiling. *Restart*: the trigger is re-run from its checkpoint.

Claim	Observation	Not excluded
Partition count changes scheduling, not the relation	property test, mode matrix, E3 (2 workers), E8	untested operator combinations; tests, not proof
Every stateful operator has a contract	registry test over the IR enums	a contract classified wrongly by hand
Buffering memory independent of input size	counting-allocator test: 1 MB at 1M and 4M rows	high-fan-out operators; receive side scales with peers
Faster than DuckDB on identical Arrow	E1 six suites; E12 intervals	warm, best-of-5, plan cache
Leads Daft and Ray Data at sf100	E7, one fleet size	fixed engine order; no scaling curve
Correct under worker loss	E8, five arms	one machine; driver loss
Repeated queries get faster	E11: all of it is the plan cache	a benefit from measured row counts, untested
The learned store reuses column statistics	E11: 4–10× steady-state cost without it	one workload
Eleven GPU workloads run as plan operators	E10, gated on agreement	random-init weights; five rows sampled for utilization

Table 9: Claim ledger: each claim with its supporting observation and what that observation does not rule out.

A Support matrix and claim ledger

Table 8 states what one engine covers by execution regime (“batch” includes morsel-parallel execution on one node), with the evidence for each row; Batcher claims nothing marked *no*. Table 9 pairs each claim with its observation and what that observation does not rule out.

B Implementation status

Evaluated here: contract dispatch and registry; the streaming executor; the Flight shuffle with byte-weighted credits and completeness manifests; replication and recovery; the plan cache and statistics reuse;

the micro-batch driver, its checkpoints and transactional sinks; the CPU-to-GPU split; inference and media expressions. *Implemented, not evaluated here*: the measured-cardinality loop since its fix; the strategy bandit’s gain; the cuDF device tier; spot-preemption migration; partition-aligned operators; lakehouse MERGE and CDC; governance rewrites. *Open defects*: device idle time in the two-stage GPU pipeline; the unreproduced 29 GB run; TPC-H q15, which failed once on a multi-node cluster (“no surviving worker to recover the join shuffle on”) and has not reproduced locally. *Future work*: dictionary and string-view kernels; a logical/physical type split; a keyed streaming state backend; driver failover; multi-stage heterogeneous scheduling; a partitioned inequality join.

C Reproducing the results

Versions. DuckDB 1.5.5, Polars 1.36.1, Daft 0.7.24, Ray 2.58.0, PySpark 4.2.0, PyArrow 19.0.1. GPU workers: torch 2.9.1+cu128, cupy 14.2, cuDF 26.06, numpy 2.2.6, Tesla T4, driver 580.126.20; model checkpoints are not revision-pinned.

Commands (B is benchmarks; $\langle s \rangle$ a suite). E1, E12: python B/run.py --benchmark $\langle s \rangle$ --engines batcher,duckdb,duckdb_arrow,polars --isolate [--repeat 5 --json-out F]. E2: --benchmark tpch --scale 10,tpcds or job, --engines batcher,duckdb. E3: python B/scenarios/claims/scheduler_equivalence.py --num-workers 2 --source D. E4, E5: B/scenarios/claims/cost_bench.py, cold_start.py. E11: B/scenarios/claims/learning_curve.py --runs 8 --arm A. E6: python B/scenarios/streaming_throughput.py --runs 5. E7: BENCH_ENGINE_TIMEOUT=400 python B/cluster/vs_ray_daft.py 100. E8: python B/cluster/robustness/worker_loss.py --rows 8000000 --keys 5000 --workers 8 --rounds 3. E9, E10: B/gpu_backend/vs_ray_daft_gpu_inference.py, B/cluster/gpu_*.py.

Identity, gate and artifact. After an in-place rebuild once swapped the engine mid-benchmark, the harness now refuses a debug build and records the SHA-256 of the extension actually loaded before and after each run. The correctness gate has also caught comparator defects: two engines folded TPC-H q6’s $0.06 + 0.01$ bound in IEEE double and dropped every row with `l_discount = 0.07`. The 2026-09-23 raw results are in `benchmarks/results/paper_2026-09-23/`; no content-addressed manifest yet binds them.